



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 8

Aim: : To Test different data quality metrics and validation techniques by using Apache Spark.

Data Quality Metrics

Metric	Description
Completeness	Are all required fields present and not null?
Uniqueness	Do identifiers (like IDs) have duplicates?
Validity	Do values conform to expected formats/ranges?
Consistency	Are related fields logically aligned? (e.g., start_date < end_date)
Accuracy	Do values match a trusted source of truth?
Timeliness	Is the data current and within acceptable time windows?
Conformity	Do categorical fields match expected values? (e.g., gender = M/F/Other)
Integrity	Are foreign key relationships respected between datasets?

Validation Techniques



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Technique	Description
Schema Validation	Ensure data types, nullability, and column names match expectations.
Custom Rules	Apply domain rules, like age ≥ 0 .
Statistical Profiling	Analyze distributions, means, counts to detect anomalies.
Regex Checks	Validate formats (e.g., email, phone).
Constraint Checks	Use unique, non-null, range constraints.
Cross-Field Checks	Validate logical relationships between fields.

Implementing with Apache Spark

Installing and setting up Apache Spark



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

Step 1: Set Up Apache Spark on Windows

Before diving into data quality testing, ensure that Apache Spark is properly set up on your Windows machine. This includes installing Java, Spark, and Python.

Step 2: Import Necessary Libraries

```
|: !pip install pyp
```

```
Requirement already satisfied: pyp in c:\users\gargi\anaconda3\lib\site-packages (3.0.9)
```

```
|: pip install pyspark
```

```
Collecting pyspark
  Using cached pyspark-3.5.5.tar.gz (317.2 MB)
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'done'
Collecting py4j==0.10.9.7 (from pyspark)
  Obtaining dependency information for py4j==0.10.9.7 from https://files.pythonhosted.org/packages/10/30/a58b32568f1623aad7db22aa9eafc4c6c55ca2726da47/py4j-0.10.9.7-py2.py3-none-any.whl.metadata
  Using cached py4j-0.10.9.7-py2.py3-none-any.whl.metadata (1.5 kB)
Using cached py4j-0.10.9.7-py2.py3-none-any.whl (200 kB)
Building wheels for collected packages: pyspark
  Building wheel for pyspark (setup.py): started
  Building wheel for pyspark (setup.py): still running...
  Building wheel for pyspark (setup.py): still running...
  Building wheel for pyspark (setup.py): finished with status 'done'
  Created wheel for pyspark: filename=pyspark-3.5.5-py2.py3-none-any.whl size=317747909 sha256=213a513ebe1a397a6ac90f2add42202b63296d279a9f8b1cb3
  Stored in directory: c:\users\gargi\appdata\local\pip\cache\wheels\0c\7f\b4\0e68c6d8d89d2e582e5498ad88616c16d7c19028680e9d3840
Successfully built pyspark
```



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

Step 3: Initialize Spark Session

```
[8]: spark = SparkSession.builder \
      .appName("DataQualityTest") \
      .getOrCreate()

print(f"Spark version: {spark.version}")
```

Spark version: 3.5.5

Step 4: Create and Load Sample Data

Let's create a sample CSV file with data quality issues for testing.

```
[9]: # First let's create a sample CSV file
with open("data.csv", "w") as f:
    f.write("id,name,age,gender\n")
    f.write("1,Alice,30,F\n")
    f.write("2,Bob,25,M\n")
    f.write("3,Charlie,35,M\n")
    f.write("4,David,,F\n") # Missing age
    f.write("5,Eve,-5,F\n") # Invalid age
    f.write("6,Frank,40,X\n") # Invalid gender
    f.write("1,Duplicate,50,M\n") # Duplicate ID

print("Sample data file created")
```

Sample data file created

1. Schema Validation

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

schema = StructType([StructField("id", StringType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True)])

df = spark.read.csv("data.csv", schema=schema)
```



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

5.1. Schema Validation

Ensure the data matches the expected schema. If there's any type mismatch, Spark will raise an error.

```
10]: schema = StructType([
    StructField("id", StringType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("gender", StringType(), True)
])

# Load data with schema validation
df = spark.read.csv("data.csv", schema=schema, header=True)
df.show()
```

```
+---+-----+---+-----+
| id|      name| age|gender|
+---+-----+---+-----+
|  1|    Alice|  30|     F|
|  2|     Bob|  25|     M|
|  3|  Charlie|  35|     M|
|  4|   David|NULL|     F|
|  5|     Eve|  -5|     F|
|  6|   Frank|  40|     X|
| 1|Duplicate|  50|     M|
+---+-----+---+-----+
```

2. Completeness & Null Checks

from pyspark.sql.types import col, count, isnan, when

```
df.select([count(when(col(c).isNull(), c)).alias(c) for c in df.columns]).show()
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

5.2. Completeness and Null Checks

Check for missing (null) values in your dataset.

```
1]: # Check for null values in each column
print("Null value counts:")
df.select([count(when(col(c).isNull(), c)).alias(c) for c in df.columns]).show()

# Calculate percentage of null values
total_rows = df.count()
for column in df.columns:
    null_count = df.filter(col(column).isNull()).count()
    null_percentage = (null_count / total_rows) * 100
    print(f"Column '{column}' has {null_percentage:.2f}% null values")
```

```
Null value counts:
+---+-----+---+-----+
| id|name|age|gender|
+---+-----+---+-----+
| 0|  0|  1|    0|
+---+-----+---+-----+
```

```
Column 'id' has 0.00% null values
Column 'name' has 0.00% null values
Column 'age' has 14.29% null values
Column 'gender' has 0.00% null values
```

3. Uniqueness Validation

```
total_count = df.count()

unique_count = df.select("id").distinct().count()

print("Uniqueness ratio:", unique_count / total_count)
```



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

▼ 5.3. Uniqueness Validation

Ensure there are no duplicate values in the id column. Calculate the ratio of unique values.

```
] : total_count = df.count()
    unique_count = df.select("id").distinct().count()
    print(f"Uniqueness ratio: {unique_count / total_count:.2f}")

    # Find duplicate IDs
    print("\nDuplicate IDs:")
    df.groupby("id").count().filter(col("count") > 1).show()
```

Uniqueness ratio: 0.86

Duplicate IDs:

```
+---+-----+
| id|count|
+---+-----+
|  1|    2|
+---+-----+
```

4. Value Range / Validity Checks

```
df.filter((col("age") < 0) | (col("age") > 120)).show()
```

▼ 5.4. Value Range / Validity Checks ¶

Perform a range check for the age column (e.g., ages should be between 0 and 120).

```
] : print("Invalid age values (not between 0 and 120):")
    df.filter((col("age") < 0) | (col("age") > 120)).show()
```

Invalid age values (not between 0 and 120):

```
+---+-----+
| id|name|age|gender|
+---+-----+
|  5| Eve| -5|      F|
+---+-----+
```

5. Conformity (e.g., gender in M/F/Other)

```
valid_genders = ["M", "F", "Other"]
```



Name: Gargi Sukhatankar
 Roll No: B027

SAP ID: 60019220125
 Branch: CSE(ICB)

5.5. Conformity Validation (e.g., gender values)

Ensure the gender column contains only valid values ("M", "F", or "Other").

```
In [ ]: valid_genders = ["M", "F", "Other"]
print("Invalid gender values:")
df.filter(~col("gender").isin(valid_genders)).show()
```

Invalid gender values:

```
+---+-----+---+-----+
| id| name|age|gender|
+---+-----+---+-----+
|  6|Frank| 40|      X|
+---+-----+---+-----+
```

```
df.filter(~col("gender").isin(valid_genders)).show()
```

6. Cross-field / Consistency Checks

```
df.filter(col("start_date") > col("end_date")).show()
```

7. Using PyDeequ for Declarative Validation

```
from pydeequ.checks import Check, CheckLevel
```

```
from pydeequ.verification import VerificationSuite
```

```
check = Check(spark, CheckLevel.Error, "Data Quality Check") \
```

```
    .isComplete("id") \
```

```
    .isUnique("id") \
```

```
    .isNonNegative("age") \
```

```
    .isContainedIn("gender", ["M", "F", "Other"])
```

```
VerificationSuite(spark).onData(df).addCheck(check).run().checkResultsAsDataFrame().show()
```

Conclusion:

Thus we successfully tested different data quality metrics and validation techniques by using Apache Spark.